

INT202W08_图

连接信息 Connectivity Information

连接信息可以由对象对 (pairs of objects) 之间存在的多种关系来定义。

图是存储、表达和利用连接信息的一种方式。

图由两部分组成: $G = (V, E)$

- 顶点集 (Vertices or nodes) : 图中对象的集合, 用 V 表示。
- 边集 (Edges) : 顶点之间的成对连接, 表示两个顶点之间的关系, 用 E 表示。

边的类型:

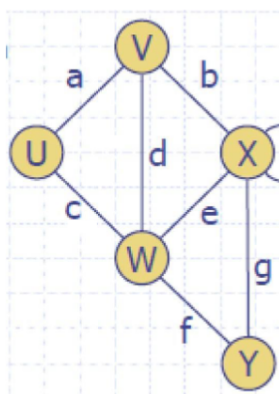
- 有向边 (directed edge)
有序顶点对 $\langle u, v \rangle$
前者是起点, 后者是终点。
- 无向边 (undirected edge)
无序顶点对 (u, v)
边没有方向。

图的类型:

有向图 (directed graph) : 所有边都是有向的

无向图 (undirected graph) : 所有边都是无向的

图的术语:



- end vertices (边的) 端点: 一条边连接的两个顶点;

如图中U和V是a的两个端点

- incident edges (顶点的) 相邻边: 一个顶点直接连接的边

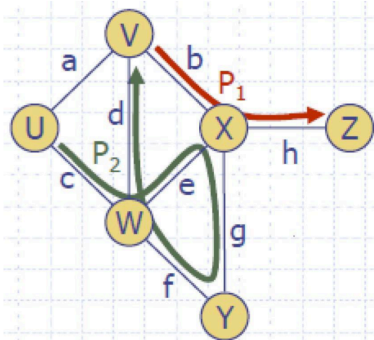
如V的相邻边是U, W, X

- adjacent vertices 相邻顶点:

一条边上的两个顶点

- degree of a vertex 顶点的度:

一个顶点上相邻边的个数

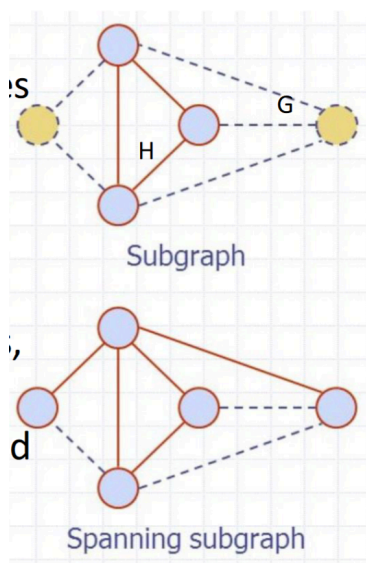


- Path 路径

顶点和边的序列, 起始和终止都是顶点

- simple path 简单路径

所有边和顶点都不重复的路径 (如P1是简单路径, P2不是)



- subgraph 子图:

图G的子图H的所有顶点和边都是G的子集

- spanning subgraph 生成子图:

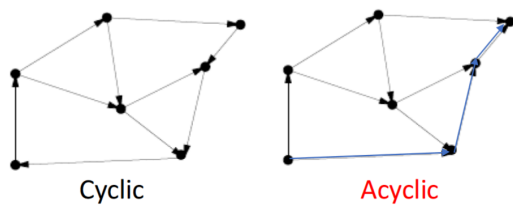
图G的生成子图S包含G的所有顶点

- (not) connected 连通性

若任意两个顶点间有连接

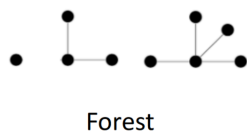
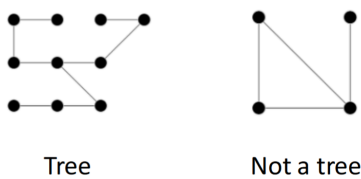
- connected components 连通分量

在图G中由边相连的顶点组成的最大联通子图



Acyclic 无环的

Cyclic 有环的



树: 连接, 且无环

森林: 无环的无向图, 不用连接, 其中每个连接部分是树

对有 n 个端点 m 条边的无向图G

- 若G是连通图, 则 $m \geq n - 1$
- 若G是树, 则 $m = n - 1$
- 若G是森林, 则 $m \leq n - 1$

生成树 (spanning tree)

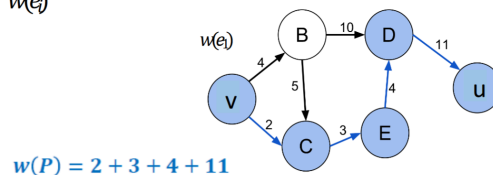
生成树不唯一除非原图是树

基于栈的非递归DFS算法：产生生成树，所有不是树的边都是back edges；用于检测环

基于队列的BFS算法：寻找图中的最短路径；到每个顶点的最短路径

Weighted Graphs加权图

The length (or weight) of a path P is the sum of the weights of the edges $e_1, \dots, e_k$ of P , i.e., $w(P) = \sum_{i=1}^k w(e_i)$



SSSP (Single-source shortest paths) 单源最短路径

对固定顶点 v ，找到从 v 到图 G 中所有其他顶点 u 的最短路径。

Dijkstra 算法

- 距离表 D ：记录从起点到其他点的最短距离
- 前置节点表 P ：记录每个节点的最短路径的上一个节点
- 优先级队列 Q ：根据节点的 D 值对节点排序，首先处理最近的未访问节点
- 最终顶点集 C ：当确认顶点具有最短路径时，添加入 C 中

Edge Relaxation松弛：当我们找到从 v 到 z 的更短路径，将 $D[z]$ 更新为这个更小的值

```
1  if  $D[u] + w(\{u, z\}) < D[z]$ 
2      then  $D[z] = D[u] + w(\{u, z\})$ 
```

不可使用的情况：

有负权重的边（哪怕为每个边添加常数）

需要贝尔曼福特算法 (bellman-ford algorithm) 是边缘松弛法，但不是贪婪算法